Sheet 03 - Convolutional Neural Networks

Introduction to Deep Learning - Summer Semester 2026

Ulf Krumnack & Robin Rawiel - Universität Osnabrück

Due: May 3, 2026

Task 1: Theory Questions [8 points]

These questions revisit the core ideas from Lecture 04 and the CNN script: images as tensors, convolution as local pattern detection, and the architectural choices that make CNNs effective for vision.

1.1 Convolution Operation [2 points]

1. Given a 5×5 input image and a 3×3 kernel, compute the output size for:
   - No padding, stride 1
   - Padding 1, stride 1
   - No padding, stride 2

   State the general formula for the output size given input size W, kernel size K, padding P, and stride S.
2. Perform **by hand** the 2D convolution (cross-correlation) of the following input with the given kernel (stride 1, no padding):

   Input:

   ```
   1 0 1 0
   0 1 0 1
   1 0 1 0
   0 1 0 1
   ```

   Kernel:

   ```
   1  0 -1
   1  0 -1
   1  0 -1
   ```

1.2 Architectural Concepts [3 points]

1. What is the **receptive field** of a neuron in a CNN? How does it grow with depth?
2. Explain the purpose of **max pooling**. What are its advantages and disadvantages?
3. What problem do **residual connections** (skip connections) solve in deep networks? Write down the residual mapping.

1.3 Images, Channels, and Parameter Efficiency [3 points]

1. Suppose we work with RGB images of size 32×32. What is the tensor shape of
   - a single image, and
   - a mini-batch of 64 such images

   in channel-first notation?

2. Consider a convolutional layer with input shape $3 \times 32 \times 32$, 16 output channels, kernel size 3×3, stride 1, and no padding.
   - How many trainable parameters does this layer have if each output channel also has one bias?
   - What is the output activation shape?
3. Suppose we flatten the same input image into a vector of length $3 \cdot 32 \cdot 32 = 3072$ and feed it into a fully connected layer with 16 output units.
   - How many trainable parameters does this fully connected layer have?
   - Briefly explain why the convolutional layer above is much more parameter-efficient.

Task 2: CNN on CIFAR-10 [12 points]

Learning objectives:

- Build a CNN architecture in PyTorch
- Train, evaluate, and visualize CNN performance
- Visualize learned filters and confusion matrix

You have already seen the main PyTorch building blocks in Practice 03. In this sheet, the goal is to combine them into one complete CNN pipeline for CIFAR-10.

2.1 Load and Explore CIFAR-10 [1 point]

As in the practice notebook, keep the model and tensors on the same `device`. The call `permute(1, 2, 0)` is only for visualization, because `matplotlib` expects channel-last images while PyTorch stores image tensors channel-first.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

CLASSES = ["plane", "car", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]
```

```
# TODO: Your solution here
```

2.2 Build the CNN [3 points]

Design a CNN with the following architecture:

Layer	Details
Conv2d	$3 \to 32$, kernel 3, padding 1
BatchNorm2d + ReLU	
Conv2d	$32 \to 32$, kernel 3, padding 1
BatchNorm2d + ReLU + MaxPool2d(2)	
Conv2d	$32 \to 64$, kernel 3, padding 1
BatchNorm2d + ReLU	

Layer	Details
Conv2d	$64 \to 64$, kernel 3, padding 1
BatchNorm2d + ReLU + MaxPool2d(2)	
Flatten + Linear($64\times8\times8 \to 256$) + ReLU + Dropout(0.5)	
Linear($256 \to 10$)	

Hint: CIFAR-10 images start at spatial size 32×32. The two `MaxPool2d(2)` layers reduce this to 16×16 and then 8×8, which is why the first linear layer receives $64 \times 8 \times 8$ input features.

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # TODO: Your solution here

    def forward(self, x):
        # TODO: Your solution here
```

2.3 Training Loop [3 points]

Remember that `model.train()` activates training-specific behavior such as dropout, while `model.eval()` switches to evaluation mode. During evaluation, `torch.no_grad()` disables gradient tracking and makes inference cheaper.

```
def train_epoch(model, loader, criterion, optimizer, device):
    """Train for one epoch. Returns (avg_loss, accuracy)."""
    # TODO: Your solution here


def evaluate(model, loader, criterion, device):
    """Evaluate the model. Returns (avg_loss, accuracy)."""
    # TODO: Your solution here
```

```
# TODO: Your solution here
```

2.4 Visualize Learned Filters [2 points]

Extract and visualize the filters from the first convolutional layer. After plotting them, briefly comment on whether some filters resemble edge detectors, color-contrast detectors, or other local pattern detectors from the lecture.

```
# TODO: Your solution here
```

2.5 Confusion Matrix [3 points]

```
# TODO: Your solution here
```

Task 3: Adversarial Examples (Bonus) [4 points]

Learning objectives:

- Implement the Fast Gradient Sign Method (FGSM)
- Understand adversarial vulnerability of neural networks
- Visualize adversarial perturbations

3.1 Load a Trained Model

Use the model trained in Task 2 (or load a pretrained one).

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Reuse model from Task 2- make sure it is in eval mode
model.eval()

# Load test data without augmentation
transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),
])
test_dataset = torchvision.datasets.CIFAR10(root="./data", train=False, download=True, transform=transfo
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64, shuffle=False)

CLASSES = ["plane", "car", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]
```

3.2 Implement FGSM [2 points]

The FGSM attack computes:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y))$$

```
def fgsm_attack(model, images, labels, epsilon, criterion):
    """Generate adversarial examples using FGSM.

    Args:
        model: Trained model.
        images: Batch of input images (requires_grad will be set).
        labels: True labels.
        epsilon: Perturbation magnitude.
        criterion: Loss function.

    Returns:
        adversarial_images: Perturbed images clamped to valid range.
    """
    # TODO: Your solution here
```

3.3 Evaluate Robustness [1 point]

Test the model's accuracy under FGSM attacks with $\epsilon \in \{0, 0.01, 0.05, 0.1, 0.2, 0.3\}$.

```
# TODO: Your solution here
```

3.4 Visualize Adversarial Examples

Show original and adversarial images side by side with their predictions.

```
# TODO: Your solution here
```

3.5 Reflection [1 point]

1. Are the perturbations visible to the human eye at $\epsilon = 0.1$?
2. Why does FGSM work - what property of neural networks does it exploit?
3. Propose one defense mechanism against adversarial examples.